

Prácticas Tema 4

Nombre de la actividad: Segunda práctica Tema 4

Contenido relacionado: Tema 4 Herencia

En esta práctica se van a asentar los conceptos de Programación Orientada a Objetos (POO) en Java, aplicándolos a una jerarquía de clases para definir figuras geométricas.

NOTA IMPORTANTE: Recuerda que es fundamental que vayas probando el código según vas implementando cada método, aunque no se pida explícitamente.

La interfaz **Figure**

En Java, una Interfaz representa las especificaciones que cumplen todas las clases que la implementan. Se utilizan habitualmente para definir el conjunto de métodos que queremos asegurarnos que esas clases proporcionen.

Por ejemplo, la interfaz Comparable representa en Java los objetos que pueden compararse entre sí. Por tanto, define el método `compareTo()` que permite comparar dos objetos. Cualquier clase cuyos objetos se puedan comparar entre sí, debería implementar este método. Así se facilita el diseño y la reutilización de código, puesto que siempre sabemos cómo se llamará el método de comparación y qué resultado devolverá.

Antes de seguir con el resto de apartados, responde a las siguientes cuestiones:

- ¿Qué es una interfaz?
- ¿Cómo se implementa en Java?
- ¿Qué significa que una clase *implementa* una interfaz? ¿Cómo se indica en java?

Todas las clases que se definan en este ejercicio deben proporcionar un conjunto de métodos básicos, como calcular el área de la figura geométrica representada. En Java, esto se implementa definiendo la interfaz que declara todos los métodos que obligatoriamente deben implementarse en dichas clases. Por tanto, todas las clases creadas en este ejercicio deben implementar la siguiente interfaz:

```
public interface Figure {  
  
    public String getTag();  
  
    public String getFigureType();  
  
    public Double area();  
  
    public void drawTxt();  
  
    public void printDescription();  
  
}
```

La figura geométrica

Una clase abstracta es una clase que no puede ser instanciada. Se indica con el modificador **abstract** en su declaración. Las clases abstractas pueden contener métodos abstractos, esto es, métodos que únicamente están declarados, pero no implementados. Estos métodos tendrán que programarse en las clases hijas. Puesto que no pueden instanciarse objetos, las clases abstractas tienen sentido como clases de las que heredan otras.

Recuerda que una clase abstracta puede tener constructores, aunque no pueden instanciarse objetos de la misma.

1. Programa la clase abstracta **GeometricFigure**, que almacenará la información común a todas las figuras geométricas (por ejemplo, una etiqueta identificativa) y proporcionará los métodos que puedan programarse independientemente de la forma concreta de la figura. Esta clase debe implementar la interfaz **Figure**. Todas las clases que representen figuras geométricas heredarán de la clase **GeometricFigure**.

Cualquier figura tendrá una etiqueta identificativa. Por tanto deberás definir un atributo de tipo `String` en esta clase para almacenar dicha etiqueta.

2. Programa un constructor que reciba como parámetro la etiqueta identificativa de la figura.
3. Programa los métodos **get** y **set** para modificar el atributo etiqueta. Puesto que estos métodos no deberían cambiar su funcionalidad en las clases hijas, decláralos como `final` para evitar que las clases hijas los sobrescriban.
4. Implementa el método **printDescription()**. Observa que no es un método abstracto, aunque invoque otros que sí lo sean. Para evitar que las clases hijas puedan modificar el formato de la descripción, decláralo como **final**.

El método debe imprimir por pantalla una descripción de la figura, incluyendo su etiqueta, el tipo de figura y su área con el siguiente formato:

```
Tag: C-5
Figure Type: Square
Area: 25
```

*Recuerda que esta clase debe implementar la interfaz **Figure**, por lo que debe proporcionar todos los métodos definidos en dicha interfaz que puedan implementarse con la información disponible en la clase: **getTag** y **printDescription**. Los métodos que no puedan implementarse, no es necesario incluirlos (Java asume automáticamente que son abstractos), pero deberán proporcionarlos las clases hijas.*

El rectángulo

La siguiente clase a implementar representará un rectángulo y, cómo no, se llamará **Rectangle**. Esta clase hereda de **GeometricFigure** e implementa la interfaz **Figure**. Para caracterizar el rectángulo se utilizarán su base y su altura, que asumiremos que son números enteros.

1. Programa la declaración de la clase y sus atributos correspondientes.
2. Programa el constructor y los métodos accesoros (**get** y **set**) básicos.
3. Programa los siguientes métodos de la clase que se especifican a continuación:

```
public String getFigureType();  
  
public Double area();  
  
public void drawTxt();
```

El método **drawTxt()** debe dibujar la figura en la consola de texto. Por ejemplo, un rectángulo con base 6 y altura 3 se podría mostrar como:

```
*****  
*****  
*****
```

4. Programa una clase **FiguresTesting** cuyo método **main** cree una instancia de la clase **Rectangle** y la muestre por consola, junto con su descripción.
5. Contesta a las siguientes cuestiones:
 - Indica la diferencia entre una clase y un objeto.
 - ¿Cómo se instancia en Java un objeto?

RECUERDA: Si una clase implementa una interfaz, automáticamente todas sus clases hijas implementan dicha interfaz, incluso aunque no se indique explícitamente en su declaración. Observa que la clase **Rectangle** implementa la interfaz **Figure**, aunque no lo indiques explícitamente en su declaración, porque hereda de una clase que lo implementa: **GeometricFigure**.

El cuadrado

Se puede decir que un cuadrado es un rectángulo con base y altura iguales. En Java, podemos crear una clase **Square** muy fácilmente, mediante herencia, a partir de la clase **Rectangle**.

1. Programa la clase **Square** de tal forma que herede de la clase **Rectangle** del apartado anterior. La clase **Square** sólo necesita un constructor que reciba dos parámetros (la etiqueta y el valor del lado), y que llame al constructor de **Rectangle** con los parámetros base y altura iguales.
2. Prueba esta nueva clase añadiendo el código necesario a la clase **FiguresTesting**.
3. Observa que gracias a la herencia, pueden invocarse los métodos programados en la clase **Rectangle** sobre un objeto de tipo **Square**, sin necesidad de reprogramarlos.
4. Contesta a las siguientes cuestiones:
 - ¿En qué consiste la herencia?
 - ¿Cómo se indica en java que una clase *hereda* de otra?
 - ¿Qué métodos de la clase padre son visibles desde la clase hija?
 - ¿En qué consiste la sobrescritura de métodos?

Recuerda que, a diferencia del resto de métodos, las clases hijas no heredan automáticamente los constructores de la clase padre, pero pueden invocarse con ayuda de **super ()**.

Fecha de límite de entrega: Una semana desde el día de trabajo en clase

Formato y espacio de entrega:

- La entrega se realizará a través de Blackboard.
- Se entregará en formato digital en un archivo comprimido en el enlace T4_Entrega2 del módulo de aprendizaje del Tema 4. **Se espera un documento pdf o en su defecto doc con el código fuente y el resultado de la consola.**
- Si es demasiado pesado para subir directamente a Blackboard, subidlo a vuestra nube de OneDrive de u-tad y compartidlo a través de Blackboard con la opción de subir archivo de almacenamiento en la nube.
- El alumno recibe un mail de confirmación cada vez que hace una entrega. Es su responsabilidad revisar que la actividad se ve correctamente (en caso de no visualizarla, contactar con el profesor).
- Sólo se permite una entrega.
- No se permitirán entregas fuera de plazo.